

- The lecture concludes by summarizing the **load factor** and the average number of probes for insertion in hash tables.
- The final load factor after all insertions is calculated, and the average number of probes is presented for the different methods discussed.

Summary of Methods:

- **Chained Hashing** (with linked lists): Simple but requires extra memory.
- **Open Addressing** (Linear, Quadratic, and Double Hashing): More memory-efficient but requires careful management of collisions and probing sequences.

The lecture provides practical techniques for handling collisions in hash tables, emphasizing how different probing strategies and secondary hashing functions can optimize performance.



more detailed explanation of all the topics with examples and visualization with elaborated break down

Detailed Explanation of Topics with Examples and Visualizations:

1. Background and Overview of Hashing

The core idea behind hashing is to map objects (e.g., integers, strings) into an array of fixed size (M) using a **hash function**. The hash function computes an index in the array where the object should be stored.

- **Goal:** Efficiently store objects such that the lookup time is constant, ideally $O(1)$.
- **Problem:** If multiple objects hash to the same index (bin), **collisions** occur. Handling these collisions is the main challenge in designing a good hash table.

2. Hash Function

A **hash function** takes an input (object) and returns a computed index in the array (i.e., a bin number).

Example: For a given object x , $\text{hash}(x)$ computes an index in the range $[0, M-1]$. The result is the bin where the object is stored.

Example 1:

If $M = 10$ and the object $x = 37$, then the hash function $\text{hash}(x) = 37 \% 10 = 7$. Therefore, the object will be stored in bin 7.

- **Challenge:** The goal is to design a hash function that distributes the objects evenly across all bins to avoid clustering.

3. Handling Collisions: Chaining with Linked Lists

In the case of a **collision**, the basic solution is to store all objects that hash to the same bin in a **linked list**. Each bin in the hash table will point to the head of the linked list. This is called **chaining**.

Example:

Let's assume we have a hash table of size $M = 5$.

We insert the following values: 2, 7, 12, 17.

1. **Insert 2:** $\text{hash}(2) = 2 \% 5 = 2$. Insert 2 into bin 2.
2. **Insert 7:** $\text{hash}(7) = 7 \% 5 = 2$. Bin 2 is already occupied, so we chain 7 to the linked list at bin 2.
3. **Insert 12:** $\text{hash}(12) = 12 \% 5 = 2$. Bin 2 is again occupied, so we chain 12 to the list.
4. **Insert 17:** $\text{hash}(17) = 17 \% 5 = 2$. Bin 2 is occupied, so we chain 17 to the list.

After insertions, bin 2 has a linked list containing $2 \rightarrow 7 \rightarrow 12 \rightarrow 17$.

4. Load Factor

The **load factor** of a hash table is defined as the number of elements (N) divided by the number of bins (M):

$$\text{Load Factor} = \frac{N}{M}$$

- **Example:** If we have 10 objects and 5 bins, the load factor is $\frac{10}{5} = 2$, meaning on average there are 2 objects per bin.

Effect on Performance:

A higher load factor leads to slower lookups. Ideally, the load factor should be kept small. If the load factor exceeds a certain threshold (e.g., 0.75), the hash table might need to be resized.

5. Problems with Chaining

While chaining solves collisions, it has some disadvantages:

1. **Memory Overhead:** Each object needs extra memory to store a pointer to the next object in the linked list.
2. **Slower Access:** In worst-case scenarios (e.g., all objects hash to the same bin), searching for an object could take $O(N)$ time because we must traverse the linked list.

Solution: One alternative to linked lists is replacing the list with a more efficient data structure, like an **AVL tree**, which can provide $O(\log(N))$ access times but comes at the cost of increased memory usage.

6. Open Addressing

In **open addressing**, all objects are stored directly in the hash table, without using linked lists. When a collision occurs, an algorithm defines how to search for the next empty bin.

Linear Probing

In **linear probing**, if an object hashes to a bin that is occupied, the algorithm checks the next bin (in a linear fashion). If that bin is also occupied, it checks the next one, and so on, wrapping around to the start of the array if necessary.

Example:

Assume a hash table of size $M = 5$.

We want to insert these values: 10, 20, 5, 15.

1. **Insert 10:** $\text{hash}(10) = 10 \% 5 = 0$. Bin 0 is empty, so 10 is placed there.
2. **Insert 20:** $\text{hash}(20) = 20 \% 5 = 0$. Bin 0 is occupied, so we check bin 1, which is empty, and insert 20 there.
3. **Insert 5:** $\text{hash}(5) = 5 \% 5 = 0$. Bin 0 is occupied, so we check bin 1 (occupied), then bin 2 (empty). We insert 5 into bin 2.
4. **Insert 15:** $\text{hash}(15) = 15 \% 5 = 0$. Bin 0, bin 1, and bin 2 are occupied, so we check bin 3 (empty) and insert 15 there.

After these insertions, the table looks like this:

```
yaml
```

```
Bin 0: 10
Bin 1: 20
Bin 2: 5
Bin 3: 15
Bin 4: empty
```

Quadratic Probing

Quadratic probing improves linear probing by using quadratic increments to find the next empty bin. The step size increases as k^2 , where k is the number of probes.

Formula:

The index to check for each probe is given by:

$$\text{bin} = (\text{initial} + k^2) \% M$$

where k starts at 1 and increases with each probe.

Example:

For a hash table of size 5, the following insertion sequence will be used:

- **Insert 10:** $\text{hash}(10) = 10 \% 5 = 0$, bin 0 is empty, so insert 10.
- **Insert 20:** $\text{hash}(20) = 20 \% 5 = 0$, bin 0 is occupied. Try bin $(0 + 1^2) \% 5 = 1$. Bin 1 is empty, so insert 20 there.
- **Insert 5:** $\text{hash}(5) = 5 \% 5 = 0$, bin 0 and bin 1 are occupied. Try bin $(0 + 2^2) \% 5 = 4$. Bin 4 is empty, so insert 5.

7. Double Hashing

Double hashing uses two hash functions: one for the initial index and another to determine the step size for probing. This reduces clustering and is often more efficient than linear or quadratic probing.

Formula:

- The initial bin is calculated using the first hash function:

$$\text{initial} = \text{hash}_1(x)$$

- The step size (or jump) is calculated using the second hash function:

$$\text{jump} = \text{hash}_2(x)$$

- The bin for each probe is calculated using:

$$\text{bin} = (\text{initial} + k \times \text{jump}) \% M$$

Example:

For a hash table with size 5, we want to insert the value `10`.

- **First Hash Function:** `hash1(10) = 10 % 5 = 0`.
- **Second Hash Function:** `hash2(10) = 3 % 5 = 3`.
- The initial bin is 0, and the jump size is 3. Therefore, the next probes will check bins 0, 3, 1, 4, and so on.

8. Visualization and Breakdown

- **Linear Probing Visualization:**

```
rust

Insert 10 -> Bin 0 (empty)
Insert 20 -> Bin 0 (occupied) -> Bin 1 (empty)
Insert 5 -> Bin 0 (occupied) -> Bin 1 (occupied) -> Bin 2 (empty)
Insert 15 -> Bin 0 (occupied) -> Bin 1 (occupied) -> Bin 2 (occupied) -> Bin 3 (empty)
```

- **Quadratic Probing:**

```
rust

Insert 10 -> Bin 0 (empty)
Insert 20 -> Bin 0 (occupied) -> Bin 1 (empty)
Insert 5 -> Bin 0 (occupied) -> Bin 1 (occupied) -> Bin 4 (empty)
```

- **Double Hashing:**

```
mathematica

Insert 10 -> First Hash: Bin 0, Second Hash: Jump size 3 -> Bin 0, 3, 1, 4, etc.
```

Conclusion

The lecture covers important techniques for handling collisions in hash tables, including:

- **Chaining** with linked lists
- **Open Addressing** (Linear, Quadratic, Double Hashing)

The choice of collision resolution method depends on memory constraints and performance requirements. Double hashing is often the most efficient solution, as it minimizes clustering and ensures a good distribution of objects in the hash table.



more detailed explanation of all the topics with perfect examples and more visualization with elaborated explanation and break down

Detailed Explanation of All Topics with Examples, Visualizations, and Breakdown:

1. Background and Goal of Hashing

Hashing is a technique used to store objects (like integers, strings, etc.) in a fixed-size array, and the goal is to map these objects to array positions (bins) as efficiently as possible. The process of mapping objects to array indices is done using a **hash function**.

Goal:

- Store objects in an array and map them to the appropriate bin (index).
 - Achieve constant time access, i.e., ideally **O(1)**, to retrieve, insert, or delete an object.
-

2. Hash Function

A **hash function** is a function that maps an input object (such as an integer or string) to a fixed-size array index. This index is called the **hash value** or **hash code**.

Example of Hash Function:

Let's say we have a hash table with a size $M = 5$ (i.e., 5 bins). We want to store the integer `37`.

- We compute $\text{hash}(37) = 37 \% 5 = 2$. Therefore, the object `37` will be stored in bin 2.

Why is a Good Hash Function Important?

- The function should evenly distribute the objects across the bins to avoid **collisions** (two objects hashing to the same bin).
 - A **bad** hash function might create many collisions, which degrades performance.
-

3. Collisions and Handling Strategies

A **collision** occurs when two objects hash to the same bin. Handling these collisions is essential for ensuring efficient operations in a hash table.

4. Handling Collisions with Chaining (Linked Lists)

Chaining is a method to handle collisions by storing all the objects that hash to the same bin in a **linked list**. Each bin will contain a pointer to the head of the linked list that stores the objects.

Example:

Let's say we have a hash table with size $M = 5$ and we want to insert the following values: `2`, `7`, `12`, `17`.

1. **Insert 2:** $\text{hash}(2) = 2 \% 5 = 2$. The object `2` is inserted into bin 2.

2. **Insert 7:** $\text{hash}(7) = 7 \% 5 = 2$. Bin 2 is occupied, so `7` is added to the linked list at bin 2.
3. **Insert 12:** $\text{hash}(12) = 12 \% 5 = 2$. Bin 2 is occupied, so `12` is added to the linked list at bin 2.
4. **Insert 17:** $\text{hash}(17) = 17 \% 5 = 2$. Bin 2 is occupied, so `17` is added to the linked list at bin 2.

After insertion, bin 2 looks like this:

```
rust
```

```
Bin 2: 2 -> 7 -> 12 -> 17
```

5. Load Factor in Hash Tables

The **load factor** is defined as the number of objects in the hash table divided by the number of bins (slots in the array).

$$\text{Load Factor} = \frac{N}{M}$$

Where:

- **N** is the number of objects in the hash table.
- **M** is the number of bins in the hash table.

Example:

- If there are 8 objects in the hash table and the hash table has 5 bins, the load factor is $\frac{8}{5} = 1.6$.
- A higher load factor indicates that there are more objects in the hash table, increasing the likelihood of collisions.

Effect on Performance:

- If the load factor is too large (close to 1), it will lead to longer lookup times, since more objects are in each bin (i.e., linked lists or probing will take longer).
- **Solution:** If the load factor exceeds a threshold (e.g., 0.75), the hash table may need to be resized (typically doubling the number of bins) and rehashed.

6. Problems with Chaining

While chaining is a simple and effective method for handling collisions, it has the following drawbacks:

1. **Extra Memory Usage:** Each object stored in a linked list requires additional memory for the pointer to the next object.
2. **Slower Access:** If many objects hash to the same bin, the time to search for a specific object can degrade to **O(N)** (where **N** is the number of elements in the linked list), especially if the hash function is poor and collisions are frequent.

7. Open Addressing (No Linked Lists)

Open addressing is an alternative to chaining. Instead of storing multiple objects in the same bin, all objects are stored directly in the hash table itself. If a collision occurs, a probing mechanism is used to

find the next available bin.

Linear Probing

In **linear probing**, when a collision occurs at bin i , the algorithm checks the next bin ($i+1$). If that bin is also occupied, it checks $i+2$, $i+3$, and so on. Once the end of the array is reached, it wraps around to the beginning of the table.

Example:

Consider a hash table of size $M = 5$, and we want to insert the integers $10, 20, 5, 15$:

1. **Insert 10:** $\text{hash}(10) = 10 \% 5 = 0$. Bin 0 is empty, so 10 is inserted.
2. **Insert 20:** $\text{hash}(20) = 20 \% 5 = 0$. Bin 0 is occupied, so we check bin 1, which is empty. 20 is inserted into bin 1.
3. **Insert 5:** $\text{hash}(5) = 5 \% 5 = 0$. Bin 0 and bin 1 are occupied, so we check bin 2, which is empty. 5 is inserted into bin 2.
4. **Insert 15:** $\text{hash}(15) = 15 \% 5 = 0$. Bin 0, bin 1, and bin 2 are occupied. We check bin 3, which is empty. 15 is inserted into bin 3.

After all insertions, the table looks like:

```
yaml
Bin 0: 10
Bin 1: 20
Bin 2: 5
Bin 3: 15
Bin 4: empty
```

Quadratic Probing

Quadratic probing reduces clustering by using quadratic increments. If a collision occurs, the algorithm checks bin $i + 1^2$, $i + 2^2$, $i + 3^2$, and so on.

Example:

Consider a hash table of size $M = 5$ with the same values $10, 20, 5, 15$:

1. **Insert 10:** $\text{hash}(10) = 10 \% 5 = 0$. Bin 0 is empty, so 10 is inserted.
2. **Insert 20:** $\text{hash}(20) = 20 \% 5 = 0$. Bin 0 is occupied. We check bin $(0 + 1^2) \% 5 = 1$, which is empty. 20 is inserted into bin 1.
3. **Insert 5:** $\text{hash}(5) = 5 \% 5 = 0$. Bin 0 and bin 1 are occupied. We check bin $(0 + 2^2) \% 5 = 4$, which is empty. 5 is inserted into bin 4.
4. **Insert 15:** $\text{hash}(15) = 15 \% 5 = 0$. Bin 0, bin 1, and bin 4 are occupied. We check bin $(0 + 3^2) \% 5 = 4$, which is occupied. Check bin $(0 + 4^2) \% 5 = 1$. The next available bin is 2, so 15 is inserted into bin 2.

After all insertions, the table looks like:

```
yaml
Bin 0: 10
Bin 1: 20
Bin 2: 15
```

```
Bin 3: empty
Bin 4: 5
```

8. Double Hashing

Double hashing uses two hash functions to determine the probe sequence. The first hash function determines the initial bin, while the second hash function calculates the step size (jump) for probing.

Formula:

- Initial bin: $\text{hash1}(x) = x \% M$
- Step size (jump): $\text{hash2}(x) = \text{some other hash function}$
- The probe sequence for the k -th probe is:

$$\text{bin} = (\text{initial} + k \times \text{jump}) \% M$$

Example:

Consider a hash table of size $M = 5$ and the values `10, 20, 5, 15`:

1. **Insert 10:** $\text{hash1}(10) = 10 \% 5 = 0$. Bin 0 is empty, so 10 is inserted.
2. **Insert 20:** $\text{hash1}(20) = 20 \% 5 = 0$. Bin 0 is occupied. Compute jump using $\text{hash2}(20) = 1$. The next bin is $(0 + 1 * 1) \% 5 = 1$, which is empty. 20 is inserted into bin 1.
3. **Insert 5:** $\text{hash1}(5) = 5 \% 5 = 0$. Bin 0 and bin 1 are occupied. Compute jump using $\text{hash2}(5) = 3$. The next bin is $(0 + 1 * 3) \% 5 = 3$, which is empty. 5 is inserted into bin 3.
4. **Insert 15:** $\text{hash1}(15) = 15 \% 5 = 0$. Bin 0, bin 1, and bin 3 are occupied. Compute jump using $\text{hash2}(15) = 2$. The next bin is $(0 + 1 * 2) \% 5 = 2$, which is empty. 15 is inserted into bin 2.

After all insertions, the table looks like:

```
yaml
Bin 0: 10
Bin 1: 20
Bin 2: 15
Bin 3: 5
Bin 4: empty
```

9. Conclusion

The lecture explains the various ways to handle collisions in hash tables, including **chaining** (with linked lists), **open addressing** (linear probing, quadratic probing, and double hashing). Each method has advantages and drawbacks:

- **Chaining** is simple but can cause slower access times if many objects hash to the same bin.
- **Open addressing** avoids extra memory usage but requires careful probing to avoid clustering.
- **Double hashing** offers the best performance by reducing clustering and ensuring good distribution across bins.

Understanding how each method works, with examples and visualizations, is key to choosing the appropriate collision resolution technique for a specific use case.

ChatGPT can make mistakes. Check important info.